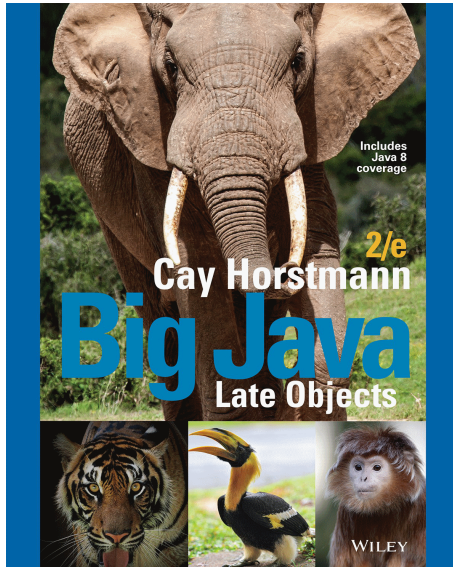


Chapter 6 – Arrays and Array Lists



Chapter Goals



- To collect elements using arrays and array lists
- To use the enhanced for loop for traversing arrays and array lists
- To learn common algorithms for processing arrays and array lists
- To work with two-dimensional arrays

Arrays

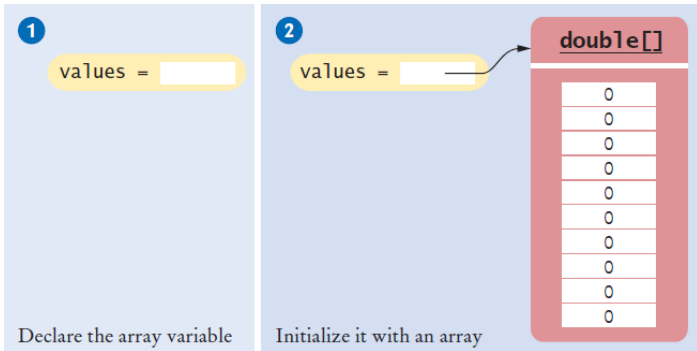
- A Computer Program often needs to store a list of values and then process them
- For example, if you had this list of values, how many variables would you need?
 - `double input1, input2, input3....`
- Arrays to the rescue!
- An array collects sequences of values of the same type

32
54
67.5
29
35
80
115
44.5
100
65

Declaring an Array

■ Declaring an array is a two step process

```
1) double[] values;           // declare array variable
2) values = new double[10];    // initialize array
```



You cannot use the array until you tell the compiler the size of the array in step 2.

Declaring and Initializing an Array

- You can declare and set the initial contents of all elements by:

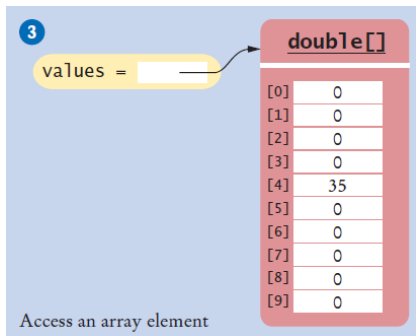
Type	Braces	Array name	Contents list	Semicolon
<code>int</code>	<code>[]</code>	<code>primes</code>	<code>= { 2, 3, 5, 7 }</code>	<code>;</code>

- You are declaring that
 - There is an array named `primes`
 - The elements inside are of type `int`
 - Reserve space for four elements
 - The compiler counts them for you!
 - Set initial values to 2, 3, 5, and 7
 - Note the curly braces around the contents list

Accessing Array Elements

- Each element is numbered
 - We call this the *index*
 - Access an element by:
 - Name of the array
 - Index number
 - `values[i]`

```
public static void main(String[] args)
{
    double values[];
    values = new double[10];
    values[4] = 35;
}
```



Syntax 6.1 Array

Syntax To construct an array: `new typeName[length]`

To access an element: `arrayReference[index]`

Diagram illustrating the syntax for array construction and access:

Name of array variable (points to `double[]`)
Type of array variable (points to `double[]`)
Element type (points to `double`)
Length (points to `10`)

```
double[] values = new double[10];  
double[] moreValues = { 32, 54, 67.5, 29, 35 };
```

Use brackets to access an element.

```
values[i] = 0;
```

The index must be ≥ 0 and $<$ the length of the array.
 See Common Error 6.1.

List of initial values

Array Index Numbers

- Array index numbers start at 0
 - The rest are positive integers
- A 10 element array has indices 0 through 9
 - **There is NO element 10!**

The first element is at index 0



```
public static void main(String[] args)
{
    double values[];
    values = new double[10];
}
```

The last element is at index 9



<u>double[]</u>	
[0]	0
[1]	0
[2]	0
[3]	0
[4]	35
[5]	0
[6]	0
[7]	0
[8]	0
[9]	0

Array Bounds Checking

- An array knows how many elements it can hold
 - `values.length` is the size of the array named `values`
 - It is an integer value (index of the last element + 1)
- Use this to range check and prevent bounds errors

```
public static void main(String[] args)
{
    int i = 10, value = 34;
    double values[];
    values = new double[10];
    if (0 <= i && i < values.length)    // length is 10
    {
        values[i] = value;
    }
}
```

Strings and arrays use different syntax to find their length:

Strings: `name.length()`

Arrays: `values.length`

Summary: Declaring Arrays

Table 1 Declaring Arrays

<pre>int[] numbers = new int[10];</pre>	An array of ten integers. All elements are initialized with zero.
<pre>final int LENGTH = 10; int[] numbers = new int[LENGTH];</pre>	It is a good idea to use a named constant instead of a “magic number”.
<pre>int length = in.nextInt(); double[] data = new double[length];</pre>	The length need not be a constant.
<pre>int[] squares = { 0, 1, 4, 9, 16 };</pre>	An array of five integers, with initial values.
<pre>String[] friends = { “Emily”, “Bob”, “Cindy” };</pre>	An array of three strings.
 <pre>double[] data = new int[10]</pre>	Error: You cannot initialize a double[] variable with an array of type int[].

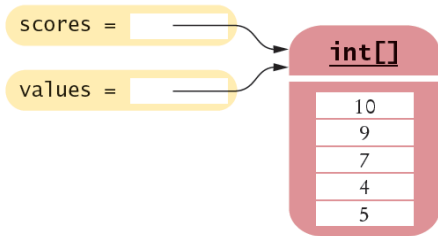
Array Aliases

- You can make one array reference refer to the same contents of another array reference:

```
int[] scores = { 10, 9, 7, 4, 5 };  
int[] values = scores; // Copying the array reference
```

References

Array contents



An array variable specifies the location of an array. Copying the reference yields a second reference to the same array.

Common Error

■ Array Bounds Errors

- Accessing a nonexistent element is very common error
- Array indexing starts at 0
- Your program will abnormally end at run time

```
public class OutOfBounds
{
    public static void main(String[] args)
    {
        double values[];
        values = new double[10];
        values[10] = 100;
    }
}
```

There is no element 10:



```
java.lang.ArrayIndexOutOfBoundsException: 10
    at OutOfBounds.main(OutOfBounds.java:7)
```

<u>double[]</u>	
[0]	0
[1]	0
[2]	0
[3]	0
[4]	35
[5]	0
[6]	0
[7]	0
[8]	0
[9]	0

Common Error

■ Uninitialized Arrays

- Don't forget to initialize the array variable!
- The compiler will catch this error

```
double[] values;  
...  
values[0] = 29.95; // Error-values not initialized
```

Error: D:\Java\Unitialized.java:7:
variable values might not have been initialized

1

values =

```
double[] values;  
values = new double[10];  
values[0] = 29.95; // No error
```

2

values =

double[]

0

The Enhanced for Loop

- Using `for` loops to 'walk' arrays is very common
 - The enhanced `for` loop simplifies the process
 - Also called the "for each" loop
 - Read this code as:
 - "For each element in the array"
- As the loop proceeds, it will:
 - Access each element in order (0 to length-1)
 - Copy it to the `element` variable
 - Execute loop body
- Not possible to:
 - Change elements
 - Get bounds error

```
double[] values = . . .;
double total = 0;
for (double element : values)
{
    total = total + element;
}
```

Syntax 6.2 The Enhanced for loop

Syntax **for** (*typeName variable : collection*)
 {
 statements
 }

This variable is set in each loop iteration.
It is only defined inside the loop.

An array

```
for (double element : values)  
{  
    sum = sum + element;  
}
```

These statements
are executed for each
element.

The variable
contains an element,
not an index.

Common Array Algorithms

- Filling an Array
- Sum and Average Values
- Find the Maximum or Minimum
- Output Elements with Separators
- Linear Search
- Removing an Element
- Inserting an Element
- Swapping Elements
- Copying Arrays
- Reading Input

Common Algorithms 1 and 2

1) Filling an Array

- Initialize an array to a set of calculated values
- Example: Fill an array with squares of 0 through 10

```
int[] values = new int[11];  
for (int i = 0; i < values.length; i++)  
{  
    values[i] = i * i;  
}
```

2) Sum and Average

- Use enhanced for loop, and make sure not to divide by zero

```
double total = 0, average = 0;  
for (double element : values)  
{  
    total = total + element;  
}  
if (values.length > 0) {  
    average = total / values.length;  
}
```

Common Algorithms 3

■ Maximum and Minimum

- Set largest to first element
- Use for or enhanced for loop
- Use the same logic for minimum

```
double largest = values[0];
for (int i = 1; i < values.length; i++)
{
    if (values[i] > largest)
    {
        largest = values[i];
    }
}
```

Typical for loop to find maximum

```
double largest = values[0];
for (double element : values)
{
    if (element > largest)
        largest = element;
}
```

Enhanced for to find maximum

```
double smallest = values[0];
for (double element : values)
{
    if (element < smallest)
        smallest = element;
}
```

Enhanced for to find minimum

Common Algorithms 4

■ Element Separators

- Output all elements with separators between them
- No separator before the first or after the last element

```
for (int i = 0; i < values.length; i++)  
{  
    if (i > 0)  
    {  
        System.out.print(" | ");  
    }  
    System.out.print(values[i]);  
}
```

32 | 54 | 67.5 | 29 | 35

■ Handy Array method: **Arrays.toString()**

- Useful for debugging!

[32, 54, 67.5, 29, 35]

```
import java.util.*;  
System.out.println(Arrays.toString(values));
```

Common Algorithms 5

■ Linear Search

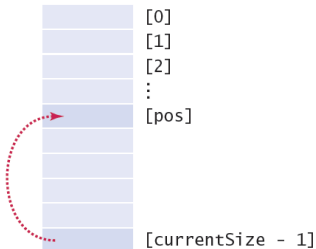
- Search for a specific value in an array
- Start from the beginning (left), stop if/when it is found
- Uses a boolean `found` flag to stop loop if found

```
int searchedValue = 100;  int pos = 0;
boolean found = false;
while (pos < values.length && !found)
{
    if (values[pos] == searchedValue)
    {
        found = true;
    }
    else
    {
        pos++;
    }
}
if (found)
{
    System.out.println("Found at position: " + pos);
}
else
{
    System.out.println("Not found");
}
```

Compound condition to prevent bounds error if value not found.

Common Algorithms 6a

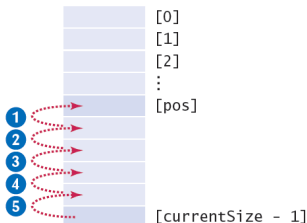
- Removing an element (at a given position)
 - Requires tracking the 'current size' (# of valid elements)
 - But don't leave a 'hole' in the array!
 - Solution depends on if you have to maintain 'order'
 - If not, find the last valid element, copy over position, update size



```
values[pos] = values[currentSize - 1];  
currentSize--;
```

Common Algorithms 6b

- Removing an element and maintaining order
 - Requires tracking the 'current size' (# of valid elements)
 - But don't leave a 'hole' in the array!
 - Solution depends on if you have to maintain 'order'
 - If so, move all of the valid elements after `pos` up one spot, update size



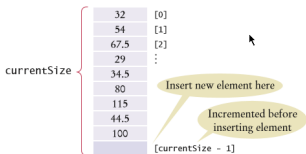
```
for (int i = pos; i < currentSize - 1; i++)  
{  
    values[i] = values[i + 1];  
}  
currentSize--;
```

Common Algorithms 7

■ Inserting an Element

- Solution depends on if you have to maintain 'order'
 - If not, just add it to the end and update the size

```
if (currentSize < values.length)
{
    currentSize++;
    values[currentSize - 1] = newElement;
}
```

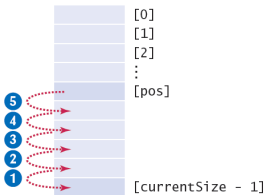


Common Algorithms 7

■ Inserting an Element

■ Solution depends on if you have to maintain 'order'

- If so, find the right spot for the new element, move all of the valid elements after 'pos' down one spot, insert the new element, and update size

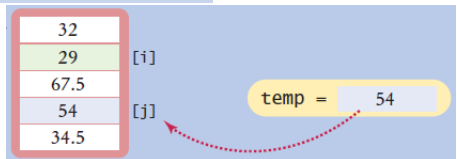
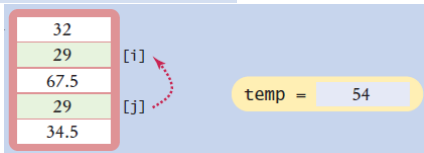
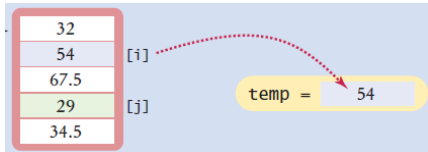


```
if (currentSize < values.length)
{
    currentSize++;
    for (int i = currentSize - 1; i > pos; i--)
    {
        values[i] = values[i - 1]; // move down
    }
    values[pos] = newElement;      // fill hole
}
```

Common Algorithms 8

■ Swapping Elements

■ Three steps using a temporary variable



```
double temp = values[i];  
values[i] = values[j];  
values[j] = temp;
```

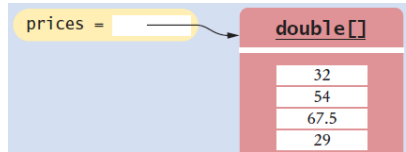
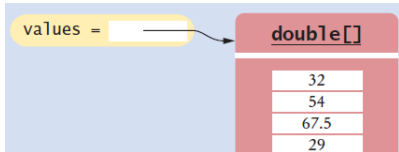
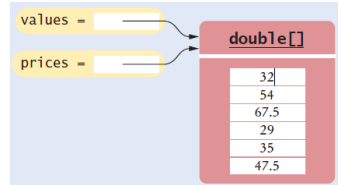
Common Algorithms 9a

■ Copying Arrays

- Not the same as copying only the reference
 - Copying creates two sets of contents!

■ Use the `Arrays.copyOf` method

```
double[] values = new double[6];  
... // Fill array  
double[] prices = values; // Only a reference so far  
prices = Arrays.copyOf(values, values.length);  
// copyOf creates the new copy, returns a reference
```



Common Algorithms 9b

- Growing an array
 - Copy the contents of one array to a larger one
 - Change the reference of the original to the larger one
- Example: Double the size of an existing array
 - Use the `Arrays.copyOf` method
 - Use `2 *` in the second parameter

```
double[] values = new double[6];  
... // Fill array  
double[] newValues = Arrays.copyOf(values, 2 * values.length);  
values = newValues;
```

`Arrays.copyOf` second parameter is the length of the new array

Common Algorithms 10

■ Reading Input

■ A: Known number of values to expect

- Make an array that size and fill it one-by-one

```
double[] inputs = new double[NUMBER_OF_INPUTS];
for (int i = 0; i < inputs.length; i++)
{
    inputs[i] = in.nextDouble();
}
```

■ B: Unknown number of values

- Make maximum sized array, maintain as partially filled array

```
double[] inputs = new double[MAX_INPUTS];
int currentSize = 0;
while (in.hasNextDouble() && currentSize < inputs.length)
{
    inputs[currentSize] = in.nextDouble();
    currentSize++;
}
```

LargestInArray.java (1)

```
1 import java.util.Scanner;
2
3 /**
4  * This program reads a sequence of values and prints them, marking the largest
5  * value.
6  */
7 public class LargestInArray
8 {
9     public static void main(String[] args)
10    {
11        final int LENGTH = 100;
12        double[] data = new double[LENGTH];
13        int currentSize = 0;
14
15        // Read inputs
16
17        System.out.println("Please enter values, Q to quit:");
18        Scanner in = new Scanner(System.in);
19        while (in.hasNextDouble() && currentSize < data.length)
20        {
21            data[currentSize] = in.nextDouble();
22            currentSize++;
23        }
24    }
25 }
```

Input values and store in next available index of the array

LargestInArray.java (2)

```
24 // Find the largest value
25
26 double largest = data[0];
27 for (int i = 1; i < currentSize; i++)
28 {
29     if (data[i] > largest)
30     {
31         largest = data[i];
32     }
33 }
34
35 // Print all values, marking the largest
36
37 for (int i = 0; i < currentSize; i++)
38 {
39     System.out.print(data[i]);
40     if (data[i] == largest)
41     {
42         System.out.print(" <== largest value");
43     }
44     System.out.println();
45 }
46 }
47 }
```

Use a `for` loop and the 'Find the largest' algorithm

Program Run

```
Please enter values, Q to quit:
35 80 115 44.5 Q
35
80
115 <== largest value
44.5
```

Special Topic: Sorting Arrays

- When you store values into an array, you can choose to either:

- Keep them unsorted (random order)

[0][1][2][3][4]

11 9 17 5 12

- Sort them (Ascending or Descending...)

[0][1][2][3][4]

5 9 11 12 17

- A sorted array makes it much easier to find a specific value in a large data set
- The Java API provides an efficient sort method:

```
Arrays.sort(values);           // Sort all of the array  
Arrays.sort(values, 0, currentSize); // partially filled
```

Special Topic: Searching

- We have seen the Linear Search (6.3.5)
 - It works on an array that is sorted, or unsorted
 - Visit each element (start to end), and stop if you find a match or find the end of the array
- Binary Search
 - Only works for a sorted array
 - Compare the middle element to our target
 - If it is lower, exclude the lower half
 - If it is higher, exclude the higher half
 - Do it again until you find the target or you cannot split what is left

Binary Search

■ Binary Search

- Only works for a sorted array
- Compare the middle element to our target
 - If it is lower, exclude the lower half
 - If it is higher, exclude the higher half
 - Do it again until you find the target or you cannot split what is left
- Example: Find the value 15

[0][1][2][3][4][5][6][7]
1 5 8 9 12 17 20 32

[0][1][2][3][4][5][6][7]
1 5 8 9 12 17 20 32

[0][1][2][3][4][5][6][7]
1 5 8 9 12 17 20 32

[0][1][2][3][4][5][6][7]
1 5 8 9 12 17 20 32

Sorry, 15 is not in this array.

Binary Search Example

```
boolean found = false; int low = 0; int pos = 0;
int high = values.length - 1;
while (low <= high && !found)
{
    pos = (low + high) / 2; // Midpoint of the subsequence
    if (values[pos] == searchedValue)
        { found = true; } // Found it!
    else if (values[pos] < searchedValue)
        { low = pos + 1; } // Look in second half
    else { high = pos - 1; } // Look in first half
}
if (found)
{ System.out.println("Found at position " + pos); }
else
{ System.out.println("Not found. Insert before position " + pos); }
```

[0][1][2][3][4][5][6][7]
1 5 8 9 12 17 20 32

[0][1][2][3][4][5][6][7]
1 5 8 9 12 17 20 32

[0][1][2][3][4][5][6][7]
1 5 8 9 12 17 20 32

Using Arrays with Methods

- Methods can be declared to receive references as parameter variables
- What if we wanted to write a method to sum all of the elements in an array?
 - Pass the array *reference* as an argument!

```
priceTotal = sum(prices);
```

↓ reference

```
public static double sum(double[] values)
{
    double total = 0;
    for (double element : values)
        total = total + element;
    return total;
}
```

prices =

double[]

32
54
67.5
29
35
47.5

Arrays can be used as method arguments and method return values.

Passing References (Step 1)

- Passing a reference gives the called method access to all of the data elements
 - It CAN change the values!
- Example: Multiply each element in the passed array by the value passed in the second parameter
- The parameter variables `values` and `factor` are created. 1

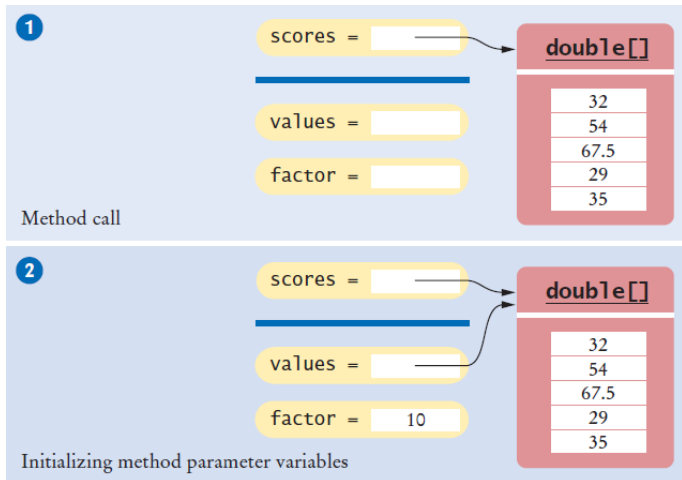
```
multiply(values, 10);
```

reference ↓

value ↓

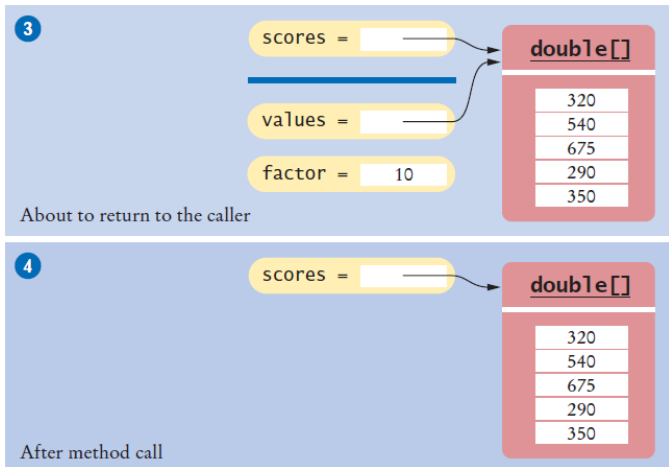
```
public static void multiply(double[] data, double factor)
{
    for (int i = 0; i < data.length; i++)
        data[i] = data[i] * factor;
}
```

Passing References (Step 2)



- The parameter variables are initialized with the arguments that are passed in the call. In our case, `values` is set to `scores` and `factor` is set to 10. Note that `values` and `scores` are references to the *same* array. **2**

Passing References (Steps 3 & 4)



- The method multiplies all array elements by 10. **3**
- The method returns. Its parameter variables are removed. However, values still refers to the array with the modified values. **4**

Method Returning an Array

- Methods can be declared to return an array

```
public static int[] squares(int n)
```

- To Call: Create a compatible array reference:

```
int[] numbers = squares(10);
```

- Call the method

value ↑

reference ↑

```
public static int[] squares(int n)
{
    int[] result = new int[n];
    for (int i = 0; i < n; i++)
    {
        result[i] = i * i;
    }
    return result;
}
```

Using Arrays with Methods

1) Decompose the task into steps

Read inputs.

Remove the minimum.

Calculate the sum.

2) Determine the algorithms to use

Read inputs.

Find the minimum.

Find the position of the minimum.

Remove the element at the position.

Calculate the sum.

3) Use methods to structure the program

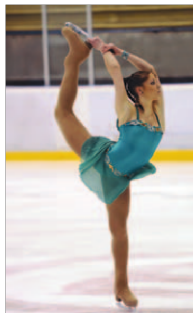
```
double[] scores = readInputs();  
double total = sum(scores) - minimum(scores);  
System.out.println("Final score: " + total);
```

4) Assemble and test the program

Multi-Dimensional Arrays

Two-Dimensional Arrays

- Arrays can be used to store data in two dimensions (2D) like a spreadsheet
 - Rows and Columns
 - Also known as a 'matrix'



	Gold	Silver	Bronze
Canada	1	0	1
China	1	1	0
Germany	0	0	1
Korea	1	0	0
Japan	0	1	1
Russia	0	1	1
United States	1	1	0

Figure 12 Figure Skating Medal Counts

Declaring Two-Dimensional Arrays

- Use two 'pairs' of square braces

```
final int COUNTRIES = 7;
final int MEDALS = 3;
int[][] counts = new int[COUNTRIES][MEDALS];
```

- You can also initialize the array

```
final int COUNTRIES = 7;
final int MEDALS = 3;
int[][] counts =
{
    { 1, 0, 1 },
    { 1, 1, 0 },
    { 0, 0, 1 },
    { 1, 0, 0 },
    { 0, 1, 1 },
    { 0, 1, 1 },
    { 1, 1, 0 }
};
```

Note the use of two 'levels' of curly braces. Each row has braces with commas separating them.

Gold	Silver	Bronze
1	0	1
1	1	0
0	0	1
1	0	0
0	1	1
0	1	1
1	1	0

Syntax 6.3 2D Array Declaration

- The name of the array continues to be a reference to the contents of the array
 - Use new or fully initialize the array

The diagram illustrates two ways to declare and initialize a 2D array in Java. The first example, `double[][] tableEntries = new double[7][3];`, is annotated with labels: 'Name' points to `tableEntries`, 'Element type' points to `double`, 'Number of rows' points to `7`, and 'Number of columns' points to `3`. A callout bubble states 'All values are initialized with 0.' The second example, `int[][] data = { { 16, 3, 2, 13 }, { 5, 10, 11, 8 }, { 9, 6, 7, 12 }, { 4, 15, 14, 1 }, };`, is annotated with 'Name' pointing to `data` and a callout bubble pointing to the list of values stating 'List of initial values'.

```
double[][] tableEntries = new double[7][3];
```

Annotations for the first example:

- Name: `tableEntries`
- Element type: `double`
- Number of rows: `7`
- Number of columns: `3`

Callout: All values are initialized with 0.

```
int[][] data = {
    { 16, 3, 2, 13 },
    { 5, 10, 11, 8 },
    { 9, 6, 7, 12 },
    { 4, 15, 14, 1 },
};
```

Annotations for the second example:

- Name: `data`
- List of initial values: The inner arrays.

Accessing Elements

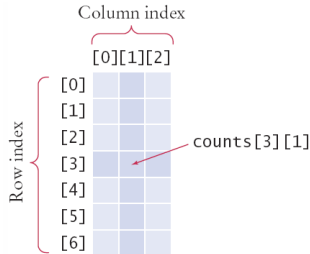
- Use two index values:

```
int value = counts[3][1];
```

Row then Column

- To print

- Use nested for loops
- Outer row(*i*) , inner column(*j*) :



```
for (int i = 0; i < COUNTRIES; i++)  
{  
    // Process the ith row  
    for (int j = 0; j < MEDALS; j++)  
    {  
        // Process the jth column in the ith row  
        System.out.printf("%8d", counts[i][j]);  
    }  
    System.out.println(); // Start a new line at the end of the row  
}
```

Medals.java (1)

```
1  /**
2   * This program prints a table of medal winner
3   * counts with row totals.
4   */
5  public class Medals
6  {
7      public static void main(String[] args)
8      {
9          final int COUNTRIES = 7;
10         final int MEDALS = 3;
11
12         String[] countries =
13             {
14                 "Canada",
15                 "China",
16                 "Germany",
17                 "Korea",
18                 "Japan",
19                 "Russia",
20                 "United States"
21             };
```

```
21
22         int[][] counts =
23             {
24                 { 1, 0, 1 },
25                 { 1, 1, 0 },
26                 { 0, 0, 1 },
27                 { 1, 0, 0 },
28                 { 0, 1, 1 },
29                 { 0, 1, 1 },
30                 { 1, 1, 0 }
31             };
```

Medals.java (2)

```
32
33     System.out.println("          Country    Gold  Silver  Bronze  Total");
34
35     // Print countries, counts, and row totals
36     for (int i = 0; i < COUNTRIES; i++)
37     {
38         // Process the ith row
39         System.out.printf("%15s", countries[i]);
40
41         int total = 0;
42
43         // Print each row element and update the row total
44         for (int j = 0; j < MEDALS; j++)
45         {
46             System.out.printf("%8d", counts[i][j]);
47             total = total + counts[i][j];
48         }
49
50         // Display the row total and print a new line
51         System.out.printf("%8d\n", total);
52     }
53 }
54 }
```

Program Run

Country	Gold	Silver	Bronze	Total
Canada	1	0	1	2
China	1	1	0	2
Germany	0	0	1	1
Korea	1	0	0	1
Japan	0	1	1	2
Russia	0	1	1	2
United States	1	1	0	2

ArrayList

Array Lists

- When you write a program that collects values, you don't always know how many values you will have.
- In such a situation, a Java `ArrayList` offers two significant advantages:
 - Array Lists can grow and shrink as needed.
 - The `ArrayList` class supplies methods for common tasks, such as inserting and removing elements.

Declaring and Using Array Lists

- The `ArrayList` class is part of the `java.util` package
 - It is a *generic* class
 - Designed to hold many types of objects
 - Provide the type of element during declaration
 - Inside `< >` as the 'type parameter'
 - The type must be a Class
 - Cannot be used for primitive types (`int`, `double...`)

```
ArrayList<String> names = new ArrayList<String>();
```

Syntax 6.4 Array Lists

■ ArrayList provides many useful methods:

- add: add an element
- get: return an element
- remove: delete an element
- set: change an element
- size: current length

Syntax To construct an array list: `new ArrayList<typeName>()`
To access an element: `arraylistReference.get(index)`
`arraylistReference.set(index, value)`

Variable type Variable name An array list object of size 0
`ArrayList<String> friends = new ArrayList<String>();`

Use the
get and set methods
to access an element.

```
friends.add("Cindy");  
String name = friends.get(i);  
friends.set(i, "Harry");
```

The add method
appends an element to the array list,
increasing its size.

The index must be ≥ 0 and $< \text{friends.size()}$.

Adding an Element with add ()

- The add method has two versions:

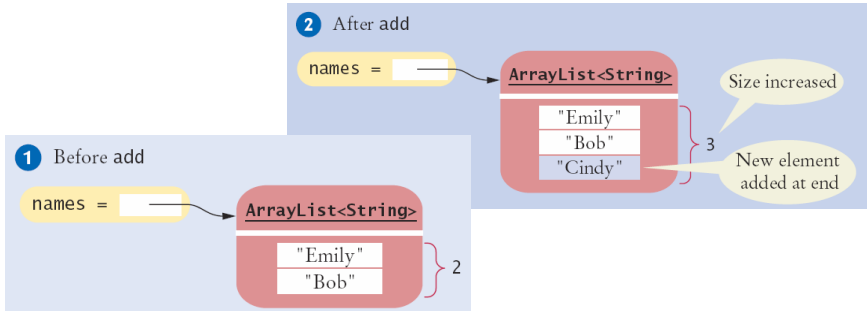
- Pass a new element to add to the end

```
names.add("Cindy");
```

- Pass a location (index) and the new value to add

```
names.add(1, "Cindy");
```

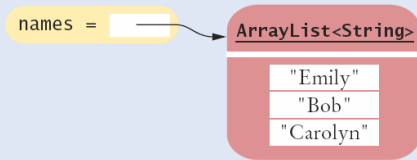
Moves all other elements



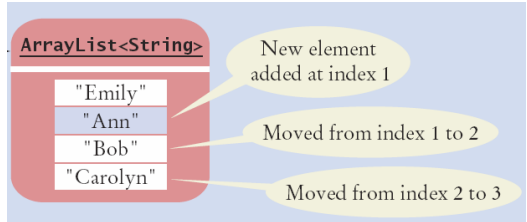
Adding an Element in the Middle

- Pass a location (index) and the new value to add
 - Moves all other elements

1 Before add

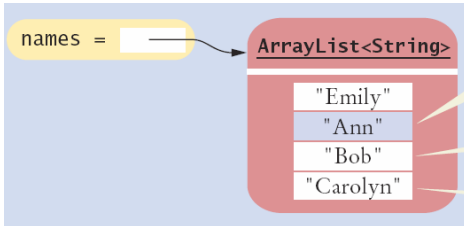


```
names.add(1, "Ann");
```

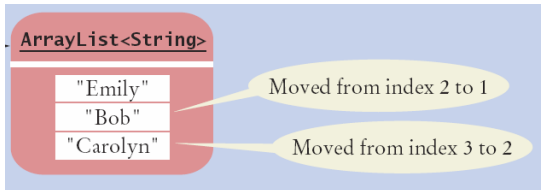


Removing an Element

- Pass a location (index) to be removed
 - Moves all other elements



```
names.remove(1);
```



Using Loops with Array Lists

- You can use the enhanced for loop with Array Lists:

```
ArrayList<String> names = . . . ;  
for (String name : names)  
{  
    System.out.println(name);  
}
```

- Or ordinary loops:

```
ArrayList<String> names = . . . ;  
for (int i = 0; i < names.size(); i++)  
{  
    String name = names.get(i);  
    System.out.println(name);  
}
```

Working with Array Lists

Table 2 Working with Array Lists

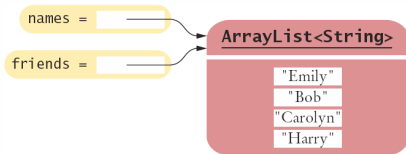
<code>ArrayList<String> names = new ArrayList<String>();</code>	Constructs an empty array list that can hold strings.
<code>names.add("Ann");</code> <code>names.add("Cindy");</code>	Adds elements to the end.
<code>System.out.println(names);</code>	Prints [Ann, Cindy].
<code>names.add(1, "Bob");</code>	Inserts an element at index 1. names is now [Ann, Bob, Cindy].
<code>names.remove(0);</code>	Removes the element at index 0. names is now [Bob, Cindy].
<code>names.set(0, "Bill");</code>	Replaces an element with a different value. names is now [Bill, Cindy].
<code>String name = names.get(i);</code>	Gets an element.
<code>String last = names.get(names.size() - 1);</code>	Gets the last element.

Copying an ArrayList

- Remember that `ArrayList` variables hold a reference to an `ArrayList` (just like arrays)

- Copying a reference:

```
ArrayList<String> friends = names;  
friends.add("Harry");
```



- To make a copy, pass the reference of the original `ArrayList` to the constructor of the new one:

reference ↓

```
ArrayList<String> newNames = new ArrayList<String>(names);
```

Array Lists and Methods

- Like arrays, Array Lists can be method parameter variables and return values
- Here is an example: a method that receives a list of `Strings` and returns the reversed list

reference ↓

```
public static ArrayList<String> reverse(ArrayList<String> names)
{
    // Allocate a list to hold the method result
    ArrayList<String> result = new ArrayList<String>();
    // Traverse the names list in reverse order (last to first)
    for (int i = names.size() - 1; i >= 0; i--)
    {
        // Add each name to the result
        result.add(names.get(i));
    }
    return result;
}
```

Wrappers and Auto-boxing

- Java provides *wrapper* classes for primitive types

- Conversions are automatic using ***auto-boxing***

- Primitive to wrapper Class

```
double x = 29.95;  
Double wrapper;  
wrapper = x; // boxing
```

wrapper =

Double

value = 29.95

- wrapper Class to primitive

```
double x;  
Double wrapper = 29.95;  
x = wrapper; // unboxing
```

Primitive Type	Wrapper Class
byte	Byte
boolean	Boolean
char	Character
double	Double
float	Float
int	Integer
long	Long
short	Short

Wrappers and Auto-boxing

- You cannot use primitive types in an `ArrayList`, but you can use their wrapper classes
 - Depend on auto-boxing for conversion
- Declare the `ArrayList` with wrapper classes for primitive types
 - Use `ArrayList<Double>`
 - Add primitive double variables
 - Or double values

```
double x = 19.95;  
ArrayList<Double> values = new ArrayList<Double>();  
values.add(29.95);           // boxing  
values.add(x);               // boxing  
double y = values.get(0);    // unboxing
```

ArrayList Algorithms

- Converting from Array to ArrayList requires changing:

- index usage: `[i]`
- `values.length`

- To

- methods: `get()`
- `values.size()`

```
double largest = values[0];
for (int i = 1; i < values.length; i++)
{
    if (values[i] > largest)
    {
        largest = values[i];
    }
}
```

```
double largest = values.get(0);
for (int i = 1; i < values.size(); i++)
{
    if (values.get(i) > largest)
    {
        largest = values.get(i);
    }
}
```

Choosing Arrays or Array Lists

- Use an Array if:
 - The size of the array never changes
 - You have a long list of primitive values
 - For efficiency reasons
 - Your instructor wants you to
- Use an Array List:
 - For just about all other cases
 - Especially if you have an unknown number of input values

Array and Array List Operations

Table 3 Comparing Array and Array List Operations

Operation	Arrays	Array Lists
Get an element.	<code>x = values[4];</code>	<code>x = values.get(4)</code>
Replace an element.	<code>values[4] = 35;</code>	<code>values.set(4, 35);</code>
Number of elements.	<code>values.length</code>	<code>values.size()</code>
Number of filled elements.	<code>currentSize</code> (companion variable, see Section 6.1.3)	<code>values.size()</code>
Remove an element.	See Section 6.3.6	<code>values.remove(4);</code>
Add an element, growing the collection.	See Section 6.3.7	<code>values.add(35);</code>
Initializing a collection.	<code>int[] values = { 1, 4, 9 };</code>	No initializer list syntax; call <code>add</code> three times.

Common Error

■ Length versus Size

- Unfortunately, the Java syntax for determining the number of elements in an array, an `ArrayList`, and a `String` is not consistent.
- It is a common error to confuse these. You just have to remember the correct syntax for each data type.

Data Type	Number of Elements
Array	<code>a.length</code>
Array list	<code>a.size()</code>
String	<code>a.length()</code>